

L2 - 6.4. Monitoring AWS Services

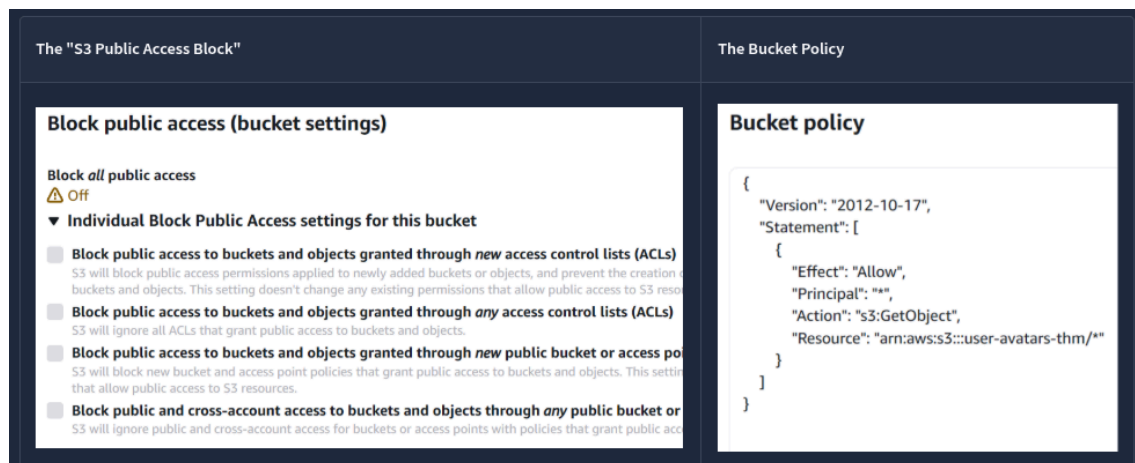
Task 1 - Introduction

Defending against all AWS attack vectors across 200+ services is difficult, but focusing SOC efforts on just 20% of the key areas can detect 80% of common attacks. This room teaches how to detect these high-impact AWS threats using CloudTrail and other cloud logs.

Task 2 - S3 Attacks and Defenses

Amazon Simple Storage Service (S3) is one of the most widely used cloud storage services. Amazon S3 is widely used across different teams—from storing room images and configuration backups to staging cloud logs for SIEMs. However, improper access controls and simple misconfigurations can expose sensitive buckets to the public, resulting in major data leaks. For example:

- [WebWork Incident\(opens in new tab\)](#): Between June 2024 and January 2025, WebWork exposed an internal S3 bucket to the public Internet. The bucket contained over 13 million records, including employee activity logs and desktop screenshots belonging to its customers.
- [WorkComposer Breach\(opens in new tab\)](#): In a similar incident, attackers accessed an unsecured, public S3 bucket operated by WorkComposer. As a result, more than 21 million sensitive desktop capture images were exposed and presumably exfiltrated.
- [Pegasus Airlines Case\(opens in new tab\)](#): In 2022, the researchers discovered 23 million files (6.5 TB of data) in a publicly-exposed S3 bucket. The data contained flight records, crew PII, proprietary source code, and various documents. The bucket remained exposed for at least a month.

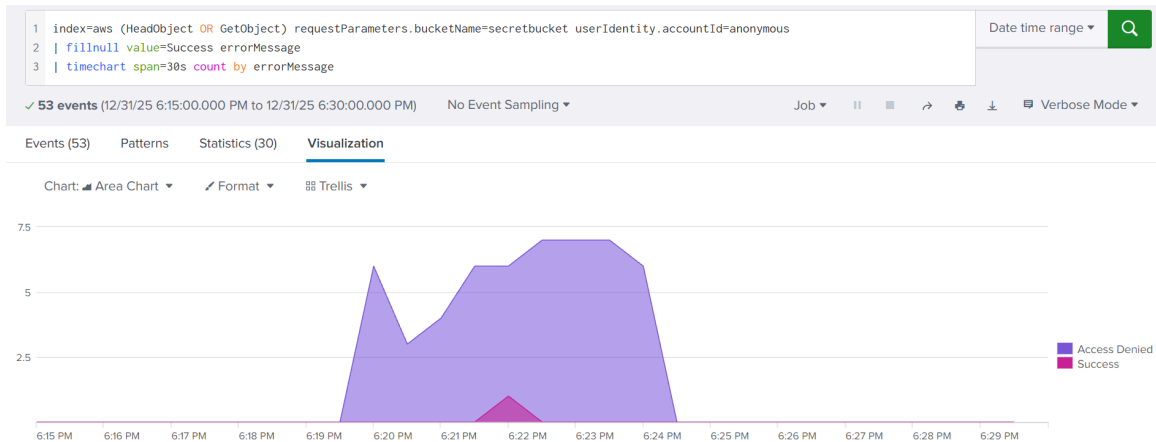


S3 buckets are private by default, making early detection of public exposure critical. Exposing a bucket requires disabling the "S3 Public Access Block" setting (`PutBucketPublicAccessBlock`) and updating access controls—primarily via modern bucket policies (`PutBucketPolicy`) rather than legacy ACLs (`PutBucketAcl`).

Not all policies with `Effect: "Allow"` and `Principal: "*"` grant public access. When hunting for public buckets via `PutBucketPolicy` events, you must analyze the full policy—additional `Deny` statements or IP-based conditions may restrict access and keep an seemingly open bucket secure.

Public read access (`GetObject` , `HeadObject` , `ListObjects`) is often intentional for hosting web assets, media, or user avatars. Security analysts can infer a bucket's purpose from its name, tags, or by directly verifying files in a browser; if ambiguity remains, consult the IT team or bucket owner.

Late Detection: If initial policy detection fails, public bucket exposures can still be identified by monitoring external object access (`GetObject` , `HeadObject`). Automated botnets will inevitably discover open buckets, generating anonymous logs from external IPs. If object listing is blocked, attackers attempting to brute-force file names will trigger a noticeable volume of "Access Denied" errors.



S3 object brute force indicated by a spike of failed GetObject requests

GuardDuty Usage: Amazon GuardDuty simplifies S3 monitoring by accurately evaluating bucket exposures. Because manual detection rules struggle to account for every policy condition, GuardDuty uses automated reasoning to reliably identify exposed buckets.

- **Policy: S3/BucketBlockPublicAccessDisabled:** The "S3 Public Access Block" feature was disabled
- **Policy: S3/BucketAnonymousAccessGranted:** The bucket was made public via ACL or bucket policy
- **Impact: S3/MaliciousIPCaller:** A known-malicious IP address accessed an exposed S3 bucket

Practice: You will investigate how Alex exposed the S3 bucket and how it led to the breach. Access the Splunk console and start your hunt from the `index=task2` query. Search for **All Time** and note that field ordering may differ from the example.

Answer the questions below

When did Alex disable the "S3 Public Access Block" feature?

2025-12-31 17:48:12

`index=task2 src_user="alex.morgan"`

click rare value:

New Search

1 index=task2 src_user="alex.morgan"

✓ 331 events (before 8/23/26 6:28:05.000 AM) No Event Sampling ▼

Events (331) Patterns Statistics

Timeline format ▼ Zoom Out

SELECTED FIELDS

- a eventName 58
- a host 1
- a source 1
- a sourcetype 1
- a start_time 100+

INTERESTING FIELDS

- a additionalEventData.AuthenticationMethod 2
- # additionalEventData.bytesTransferredIn 7
- # additionalEventData.bytesTransferredOut 25

eventName

58 Values, 100% of events

Selected Yes No

Reports

Top values Top values by time **Rare values**

Events with this field

Top 10 Values

Value	Count	%
ListManagedNotificationEvents	57	17.22%
ValidatePolicy	39	11.782%
ListFunctions20150331	34	10.272%
GetAccountPublicAccessBlock	27	8.157%
GetBucketPublicAccessBlock	27	8.157%
GetBucketVersioning	11	3.323%
GetBucketOwnershipControls	9	2.719%
GetBucketPolicy	9	2.719%
DescribeTrails	8	2.417%
GetTrailStatus	8	2.417%

then select `PutBucketPublicAccessBlock` add to the query `| table _time`

What is the SID of the applied policy that made the bucket public?

TempAccessDeniedDebug

```
index=task2 src_user="alex.morgan" "requestParameters.bucketName"="thm-internal-backups" eventName=PutBucketPolicy
```

search under the statement section:

```
a requestParameters.bucketPolicy.Statement().Effect 1
a requestParameters.bucketPolicy.Statement().Principal 1
a requestParameters.bucketPolicy.Statement().Resource 1
a requestParameters.bucketPolicy.Statement().Sid 2
a requestParameters.bucketPolicy.Version 1
a requestParameters.Host 1
a requestParameters.policy 1
# resources().accountId 1
a resources().ARN 1
a resources().type 1
a responseElements 1
a signature 1
```

requestParameters.bucketPolicy.Statement().Sid

2 Values, 100% of events

Selected Yes No

Reports

Top values Top values by time Rare values

Events with this field

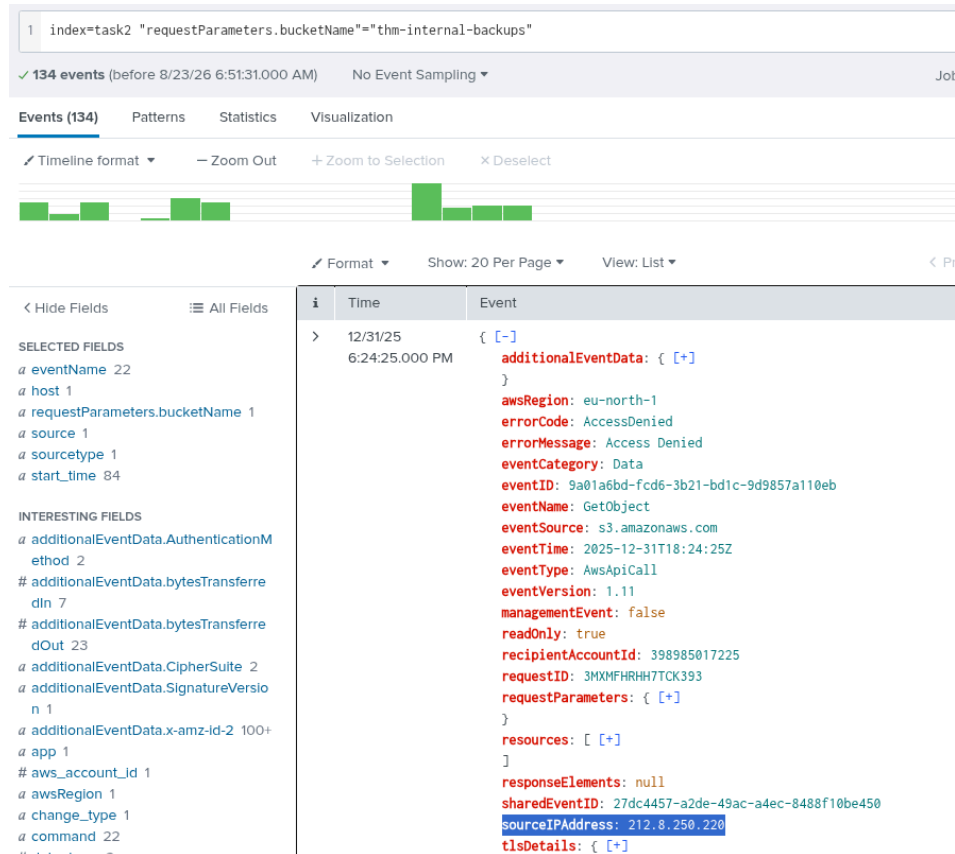
Values

Value	Count	%
AllowAccessOnlyFromTorontoOffice	1	50%
TempAccessDeniedDebug	1	50%

Which IP address started the bucket scan soon after it was exposed?

212.8.250.220

```
index=task2 "requestParameters.bucketName"="thm-internal-backups"
```

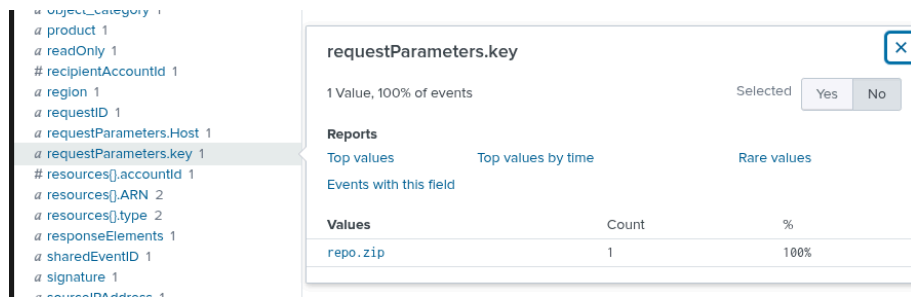


How many filenames were attempted, and which file was exfiltrated?

filter `requestParameters.key`, it shows 53 values

```
index=task2 "requestParameters.bucketName"="thm-internal-backups" src_ip="212.8.250.220" | table _time requestParameters.key
```

Shows 54 values but one of them is empty, for this reason the value is 53. Filter message, click success. Then filter `requestParameters.key`



Task 3 - EC2 Internet Exposure

EC2 Internet Exposure: Amazon EC2 instances rely on attached AWS security groups (SGs) to control inbound access, such as enabling traditional SSH or RDP connections. An SSH or HTTP rule configured within a security group can be applied across single or multiple EC2 instances simultaneously.

Early Detection: VM breaches frequently stem from opening access ports without adequate safeguards, such as using weak passwords, exposing SSH keys, or hosting vulnerable web applications. Prompt detection requires alerting immediately when a security group is modified to permit public access (like RDP) or when a new EC2 instance is launched using an insecure security group:



It is sometimes hard to correlate security groups and resources, such as EC2 instances, with the locations where they are used. Unless you have AWS access or SOAR automations that can do it for you, you need to perform an additional investigation. Below are two playbooks you can use to hunt for insecure security group changes and identify instances that were affected:

Hunt 1. A new security group has been created

1	Review the group creation event and note its name and ID (requestParameters and responseElements fields)	<code>index=* eventName=CreateSecurityGroup</code>
2	Review the security group for any insecure inbound rules (The rules are kept in the requestParameters field)	<code>index=* eventName=AuthorizeSecurityGroupIngress sg-xxxxxx</code>
3	Check if any new or old VMs start using this security group (The instance ID is kept in the responseElements field)	<code>index=* eventName IN(RunInstances, ModifyInstanceAttribute) sg-xxxxxx</code>
4	If the rules are insecure or inappropriate for the linked VMs, reach out to Alice and ask her to harden the security group	N/A

Hunt 2. One of the security group rules has been modified

1	Review the changes and note the security group ID (The rules are kept in the requestParameters field)	<code>index=* eventName IN(*SecurityGroupIngress, ModifySecurityGroupRules)</code>
2	Find the group creation and note its author and description (Search 90+ days back in case you don't see results)	<code>index=* earliest=-1y eventName=CreateSecurityGroup sg-xxxxxx</code>
3	Identify the EC2 instances that use the modified security group	<code>index=* eventName IN(RunInstances, ModifyInstanceAttribute) sg-xxxxxx</code>

	(Note: Services like RDS rely on the security groups, too)	
4	If the changes are insecure or inappropriate for the linked VMs, reach out to Bob and ask him to harden the security group	N/A

Determining actual VM exposure can be complex due to overlapping AWS network controls, including Security Groups, Network ACLs, and host-level VM firewalls. Because secondary controls may keep an apparently exposed instance private, analysts should verify port accessibility directly with a scanner or confirm with IT before escalating an incident.

Late Detection: If you don't alert on insecure security group changes, such as exposing SSH, RDP, or VNC, you are leaving an Internet-facing entry point for attackers to exploit. Even if detections miss the misconfiguration, a mature SOC team should still apply defense in depth, for example:

- Alert on insecure security group modifications with CloudTrail, as described in this task
- Enable GuardDuty, which may generate the **Recon: EC2/PortProbeUnprotectedPort** finding
- Collect **auth.log** from the VMs and detect SSH brute force from the endpoint point of view
- As a last resort, detect the dropped miner with installed EDR, network, or endpoint logs

Practice: In this task, you will investigate a typical EC2 exposure scenario. Continue with the VM and use this Splunk query: `index=task3`. Search for **All Time** and start your hunt from the earliest logs.

Answer the questions below

Which security group did Emma create, and which risky service did it expose?
website-access-sg, SSH


```
index=task3 eventName=CreateSecurityGroup
```

check: requestParameters.groupDescription : website-access-sg

we got sg value, filter object="sg-088dacb4d53945be6"

```
index="task3" eventName=AuthorizeSecurityGroupIngress sg-088dacb4d53945be6
```

investigare risky port from

responseElements.securityGroupRuleSet.items{}.fromPort

The screenshot shows the AWS CloudTrail console with a filter applied to the event 'AuthorizeSecurityGroupIngress'. The filter is on the field 'responseElements.securityGroupRuleSet.items{}.fromPort' with a value of '22'. A modal window titled 'responseElements.securityGroupRuleSet.items{}.fromPort' is open, showing '3 Values, 100% of events'. The modal includes a 'Reports' section with three tabs: 'Top values', 'Top values by time', and 'Rare values'. The 'Top values' tab is selected, displaying a table with the following data:

Values	Count	%
22	1	100%
443	1	100%
80	1	100%

Which EC2 instance ID was created shortly after and uses that security group?

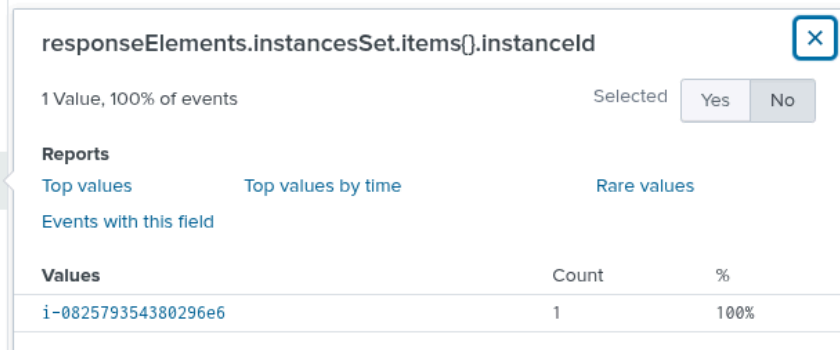
i-082579354380296e6

```
index=task3 eventName IN(RunInstances, ModifyInstanceAttribut  
e) sg-088dacb4d53945be6
```

instance ID = object ID i-082579354380296e6

or filter: response.Elements.instancesSet.items.InstanceId

```
a responseElements.instancesSet.item  
s().groupSet.items().groupName 1  
a responseElements.instancesSet.item  
s().hypervisor 1  
a responseElements.instancesSet.item  
s().imageId 1  
a responseElements.instancesSet.item  
s().instanceId 1  
# responseElements.instancesSet.item  
s().instanceState.code 1  
a responseElements.instancesSet.item  
s().instanceState.name 1  
a responseElements.instancesSet.item  
s().instanceType 1
```



responseElements.instancesSet.items().instanceId

1 Value, 100% of events Selected Yes No

Reports

Top values Top values by time Rare values

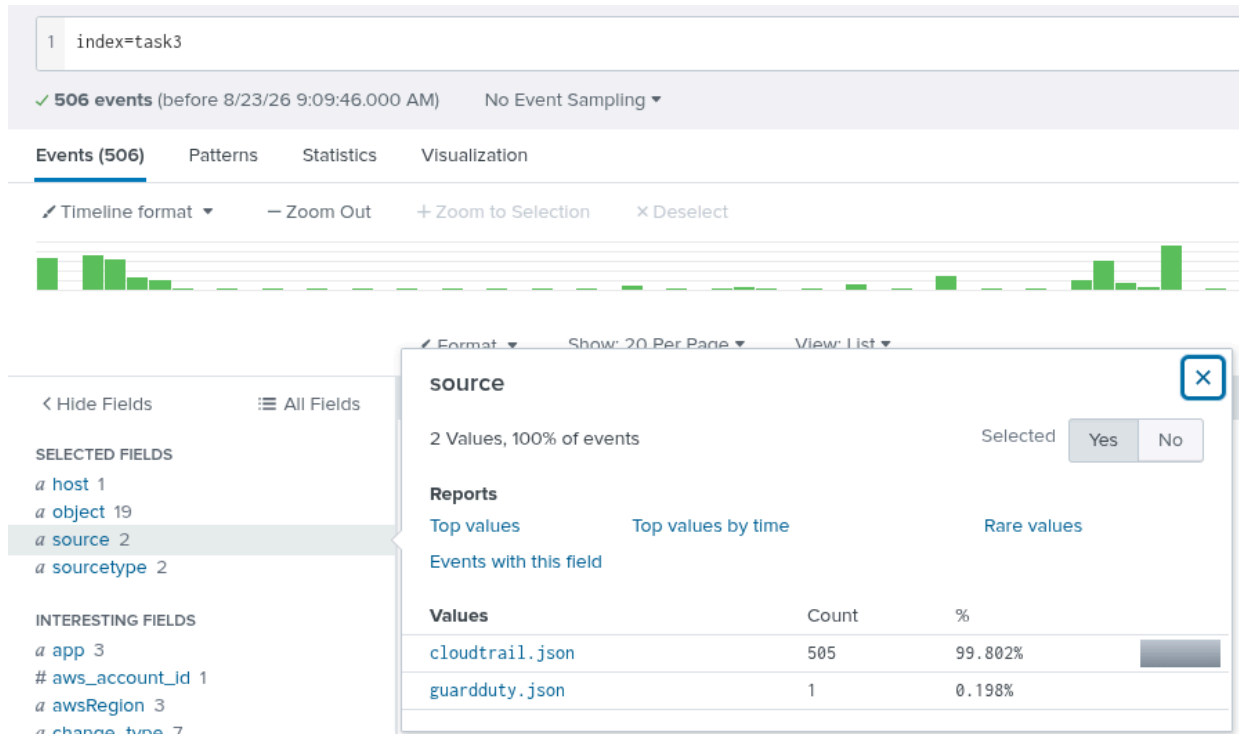
Events with this field

Values	Count	%
i-082579354380296e6	1	100%

According to the GuardDuty alert, which IP soon attacked the instance?

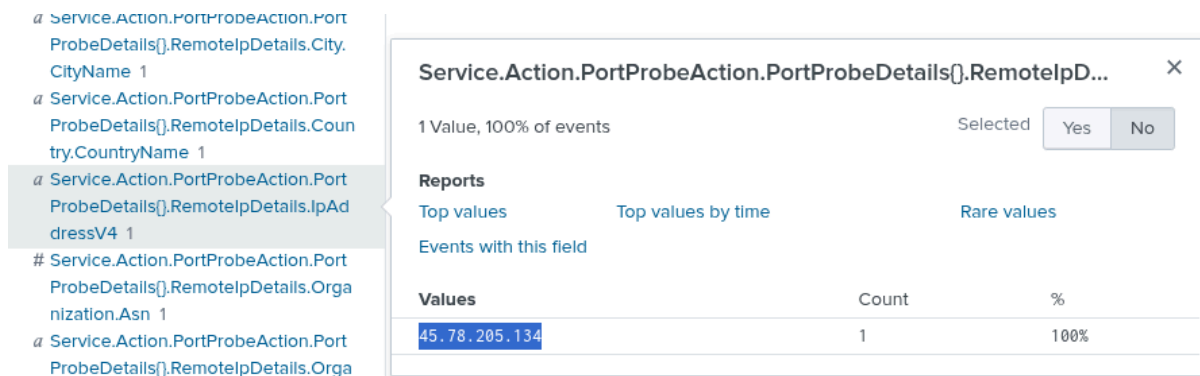
45.78.205.134

filter source guardduty.json



filter:

Service.Action.PortProbeAction.PortProbeDetails{}.RemoteIpDetails.IpAddressV4



When did Emma revoke the insecure rule from the security group?

2025-12-31 21:58:34

filter eventName from task3

index=task3 | rare limit=20 eventName

filter: RevokeSecurityGroupIngress

New Search

1
index=task3 eventName=RevokeSecurityGroupIngress

✓ 1 event (before 8/23/26 9:22:06.000 AM)
No Event Sampling ▼

Events (1)
Patterns
Statistics
Visualization

Timeline format ▼
Zoom Out
Zoom to Selection
Deselect

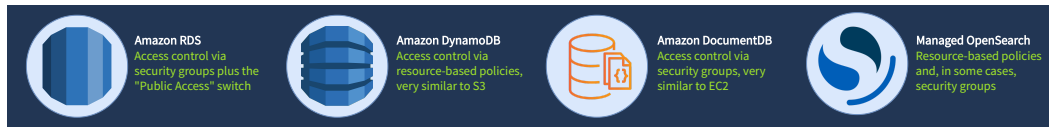
Format ▼
Show: 20 Per Page ▼
View: List ▼

< Hide Fields		≡ All Fields	i	Time	Event
SELECTED FIELDS a host 1 a object 1 a source 1 a sourcetype 1 INTERESTING FIELDS a action 1 a app 1 # aws_account_id 1 a awsRegion 1 a change_type 1 a command 1 # date_hour 1 # date_mday 1 # date_minute 1 a date_month 1 # date_second 1 a date_wday 1 # date_year 1			>	12/31/25 9:58:34.000 PM	{ [-] awsRegion: us-east-2 eventCategory: Management eventID: f98d3b73-d821-44c9-ba2b-927a9ec52b07 eventName: RevokeSecurityGroupIngress eventSource: ec2.amazonaws.com eventTime: 2025-12-31T21:58:34Z eventType: AwsApiCall eventVersion: 1.11 managementEvent: true readOnly: false recipientAccountId: 398985017225 requestID: 455cb0a8-1488-4cf1-b1b6-1cadf83eb69e requestParameters: { [+] } responseElements: { [+] } sessionCredentialFromConsole: true sourceIPAddress: 91.66.40.82 tlsDetails: { [+] } }

Task 4 - Risks of Public Databases

Exposed Database: Numerous AWS database services—such as Amazon RDS, DynamoDB, Redshift, and OpenSearch—can be exposed to the internet. Because each service manages public access uniquely, detection strategies vary; however, public vs. private visibility is generally governed by two main categories of security controls:

- **Access Policies:** Resource-based policies that define service access, same as with S3
- **Network Controls:** VPCs, subnets, access control lists, security groups, same as with EC2



Amazon RDS Example: Amazon RDS is a managed relational database service that supports many engines, such as MySQL and PostgreSQL. It is the most common place where customer emails, passwords, and credit cards are stored. To make a grave mistake, make the RDS instance public, and welcome threat actors for data exfiltration, you'd need to enable the "Public Access" switch and allow inbound access from 0.0.0.0/0 in the DB's security group. Here is how it looks in logs:

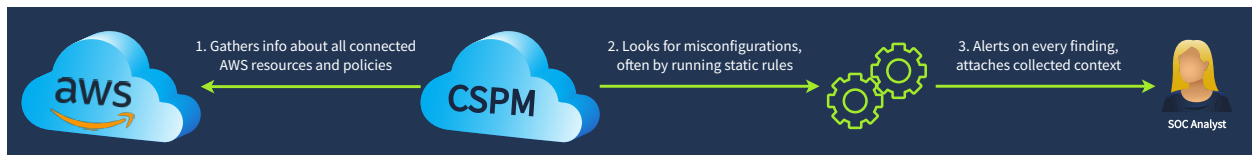


Note that, same as with EC2 instances, a database may be private at first but become public months or years later due to an accidental misconfiguration (**ModifyDBInstance**) or insecure change in its security groups (***SecurityGroupIngress, ModifySecurityGroupRules**). Also, keep in mind that exposure can occur at creation time in a single **CreateDBInstance** event, where you can enable "Public Access" on the spot and attach the instance to an existing security group.

SOC Team and CSPM: Detecting misconfigurations across services like S3, EC2, and RDS involves complex edge cases that make manual log-based hunting difficult for routine SOC operations. While DFIR teams use these detailed techniques for deeper investigation, SOCs streamline day-to-day exposure monitoring using Cloud Security Posture Management (CSPM) tools—such as AWS Config, Wiz, and Lacework—to continuously automate configuration tracking and compliance checks.

1. Continuously scan your AWS environment for exposures and misconfigurations
2. Detect if any deployed resources are exposed, vulnerable, or misconfigured
3. Optionally, revert the misconfiguration without waiting for SOC involvement
4. Generate an alert with more context than with independent logs

CloudTrail



Real-World Scenario: Database exposures are rare but carry devastating consequences. In one real-world case, a payment provider migrated an OpenSearch cluster holding customer bank transactions without carrying over security settings, leaving it publicly accessible. Within a week, adversaries exfiltrated the sensitive data, extorted the company, and triggered severe regulatory fines—all from a single misconfiguration. To prevent this as a SOC analyst, you had to:

- Develop detection rules that detect **any** change to the most critical OpenSearch clusters
- Develop detection rules that cover an insecure change to OpenSearch policies and SGs
- Configure a CSPM solution that can detect and immediately prevent cluster exposure

Practice: In this task, you will analyze the RDS database exposure scenario. Continue with the VM and use this Splunk query: `index=task4`. Search for **All Time** and focus on the write events (`readOnly=false`).

Answer the questions below

| What is the name (instance identifier) of the created RDS instance?

db-thm-preprod-qa

run the Query: `index=task4 readOnly=false`

Select Fields

Select All Within Filter Deselect All Coverage: 1% or more ▾

i	✓ ▾	Field ⇅
>	☐	requestParameters.dbInstanceClass
▼	☐	requestParameters.dbInstanceIdentifier
Reports		
Top values Events with this field		
Top values by time		
Rare values		
db-thm-preprod-qa		36

add to the query `eventName=CreateDBInstance` , filter
`requestParameters.dbInstanceIdentifier`

Which two events indicate the database is Internet-exposed?

`dcb54877, 0a3b23c1`

```
index="task4" readOnly=false db-thm-preprod-qa
```

add `eventName` to the query:

```

a date_wday 1
# date_year 1
# date_zone 1
a dest 1
a dvc 1
a errorCode 1
a eventCategory 1
a eventID 2
a eventName 2
a eventSource 1
a eventTime 2
a eventType 1
a eventtype 1
# eventVersion 1
a index 1
# linecount 1
a managementEvent 1
a msg 1

```

```

}
sessionCredentialFromConsole

```

eventName

2 Values, 100% of events

Reports

[Top values](#)
[Top values by time](#)

Events with this field

Values	Count
CreateDBInstance	1
ModifyDBInstance	1

eventName: CreateDBInstance

eventSource: rds.amazonaws.com

```
index="task4" readOnly=false db-thm-preprod-qa eventName=ModifyDBInstance
```


splunk>enterprise

Apps ▾

Search

Analytics

Datasets

Reports

Alerts

Dashboards

New Search

1 index="task4" readOnly=false db-thm-preprod-qa eventName=ModifyDBInstance

✓ 1 event (before 8/23/26 3:03:58.000 PM)

No Event Sampling ▾

Events (1)

Patterns

Statistics

Visualization

✎ Timeline format ▾

— Zoom Out

+ Zoom to Selection

✕ Deselect

Format ▾

Show: 20 Per Page ▾

View: List ▾

< Hide Fields

≡ All Fields

SELECTED FIELDS

a host 1

a source 1

a sourcetype 1

INTERESTING FIELDS

a app 1

aws_account_id 1

a awsRegion 1

a change_type 1

a command 1

date_hour 1

date_minute 1

i	Time	Event
>	1/2/26 6:10:37.000 PM	<pre>{ [-] awsRegion: eu-central-1 eventCategory: Management eventID: dcb54877-b370-453b-ac1a-a1f124101765 eventName: ModifyDBInstance eventSource: rds.amazonaws.com eventTime: 2026-01-02T18:10:37Z eventType: AwsApiCall eventVersion: 1.11 managementEvent: true readOnly: false recipientAccountId: 398985017225 requestID: 421e99d8-d731-453b-88af-ea7f2596d960 }</pre>

New Search

1
index=task4 ModifySecurityGroupRules

✓ 1 event (before 8/23/26 3:11:32.000 PM)
No Event Sampling ▼

Events (1)
Patterns
Statistics
Visualization

Timeline format ▼
Zoom Out
Zoom to Selection
Deselect

Format ▼
Show: 20 Per Page ▼
View: List ▼

< Hide Fields
All Fields

SELECTED FIELDS
a host 1
a source 1
a sourcetype 1

INTERESTING FIELDS
a app 1
aws_account_id 1
a awsRegion 1
a change_type 1
a command 1
date_hour 1
date_mday 1
date_minute 1
a date_month 1
date_second 1

i	Time	Event
>	1/2/26 6:13:30.000 PM	{ [-] awsRegion: eu-central-1 eventCategory: Management eventID: 0a3b23c1-b76e-4c6c-b677-e0c27b0adfee eventName: ModifySecurityGroupRules eventSource: ec2.amazonaws.com eventTime: 2026-01-02T18:13:30Z eventType: AwsApiCall eventVersion: 1.11 managementEvent: true readOnly: false recipientAccountId: 398985017225 requestID: 9f1a122f-013d-4221-90cf-703be7c497f8 requestParameters: { [+] } responseElements: { [+] }

Task 5 - Detecting Cloud Discovery

Cloud Discovery Intro: While Initial Access and Impact occur in all completed intrusions, complex attacks often require an intermediate phase: Discovery. When immediate impact is blocked by strict permissions or controls, adversaries map the cloud environment to identify vulnerable paths and assets for further exploitation.

Initial Discovery: Targeted AWS attacks typically start with initial discovery ("whoami") commands following access. While AWS Management Console activity generates automated noise that blends with normal user actions, CLI or SDK-based reconnaissance generates distinct API calls—making programmatically driven discovery far easier to detect in cloud logs.

Initial Discovery Commands (Output Simplified)

```
// GetCallerIdentity: AWS alternative for "whoami"
attacker@kali:~$ aws iam get-caller-identity
{
  "UserId": "AIDAVZZK4G6EVLNRWXXX",
  "Account": "398985017225",
  "Arn": "arn:aws:iam::398985017225:user/john.doe"
}

// ListAttachedUserPolicies: Verifying own privileges
attacker@kali:~$ aws iam list-attached-user-policies --user-name john.doe
[
  {
    "PolicyName": "AdministratorAccess",
    "PolicyArn": "arn:aws:iam::aws:policy/AdministratorAccess"
  }
]

// GetAccountSummary: Listing AWS account statistics
attacker@kali:~$ aws iam get-account-summary
{
  "Policies": 12,
  "Users": 16,
  "Providers": 1,
  "AccountMFAEnabled": 4,
  "AccountAccessKeysPresent": 1,
  "AccountPasswordPresent": 1,
  [...]
}
```

After initial "whoami" checks (`GetUser` , `GetCallerIdentity`), policy enumeration (`ListUserPolicies`), and account context gathering (`GetAccountSummary`), attackers proceed to detailed IAM and service discovery. However, adversaries who already understand their target's infrastructure and privileges may skip this early discovery phase altogether.

Broad Discovery: Opportunistic threat actors often run broad scans across key AWS services—such as listing S3 buckets (`ListBuckets`), EC2 instances (`DescribeInstances`), Lambda functions (`ListFunctions*`), or IAM users (`ListUsers`). SOC teams can effectively detect CLI- and SDK-driven reconnaissance by setting up alert rules that flag sudden spikes in distinct API calls across multiple services from a single identity. The rule below is a starting point for detecting discovery activity originating from the AWS CLI or SDK:

```

1 index=task5
2 NOT src_ip=*.amazonaws.com
3 NOT userIdentity.sessionContext.attributes.creationDate=*
4 | bin span=10m _time
5 | stats dc(eventName) as uniqevt dc(eventSource) as uniqsvc
6 | values(eventName) as events values(eventSource) as services
7 | by userName, user_access_key, _time
8 | where uniqevt >= 10 OR uniqsvc >= 5
9 | table _time userName user_access_key uniqevt uniqsvc events services
  
```

12 events (1/4/26 11:00:00.000 AM to 1/5/26 11:05:27.000 AM) No Event Sampling

Events (12) Patterns Statistics (1) Visualization

Show: 20 Per Page Format Preview: On

_time	userName	user_access_key	uniqevt	uniqsvc	events	services
2026-01-05 08:40:00	jose.martinez	AKIAVZZK4G6E27FHEW02	9	7	DescribeDBInstances DescribeInstances DescribeTrails GetAccountSummary GetCallerIdentity ListAttachedUserPolicies ListBuckets ListFunctions20150331 ListUsers	cloudtrail.amazonaws.com ec2.amazonaws.com iam.amazonaws.com lambda.amazonaws.com rds.amazonaws.com s3.amazonaws.com sts.amazonaws.com

Deeper Discovery: After locating key assets during initial scans, adversaries perform deeper discovery on specific targets—such as querying `GetUser` or `ListAccessKeys` for a high-value IAM user. In large environments, attackers often filter resource queries using terms like "secret" or "vault" to quickly pinpoint critical systems, which can be tracked in CloudTrail logs under the `requestParameters` field.

```

// Three detection opportunities from a single event
{
  "userIdentity": {
    "userName": "john.doe"
  },
  "eventTime": "2025-12-31T10:30:45Z",
  "eventSource": "ec2.amazonaws.com",
  "eventName": "DescribeInstances",
  "requestParameters": {
    "InstanceIds": ["i-1234567890"]
  }
}
  
```

```

    "awsRegion": "us-east-1",
    "sourceIPAddress": "167.99.40.20",
    // #1 Known-malicious IP!
    "userAgent": "aws-cli/1.22.34 Linux/5.15.0-kali3-amd64
[...]", // #2 Kali Linux user-agent!
    "requestParameters": {
        "instancesSet": {},
        "filterSet": {
            "items": [
// #3 Suspicious search keywords!
            {
                "name": "tag:Name",
                "valueSet": {
                    "items": [{ "value": "*vault*" }, { "value": "*se
cret*" }]
                }
            }
        ]
    }
}

```

Practice: You will analyze a simple Discovery originating from a breached IAM user. Search for **All Time** and start your search from the `index=task5` query.

Answer the questions below

| What was the second Discovery command the adversary ran?

ListAttachedUserPolicies

run the query: `index=task5 then filter eventName`

```

index=task5
| table _time eventName

```

_time	eventName
2026-01-05 08:48:21	GetCallerIdentity
2026-01-05 08:48:31	ListAttachedUserPolicies
2026-01-05 08:48:43	GetAccountSummary
2026-01-05 08:48:55	ListUsers
2026-01-05 08:49:15	DescribeTrails
2026-01-05 08:49:22	ListFunctions20150331
2026-01-05 08:49:26	DescribeDBInstances
2026-01-05 08:49:31	DescribeInstances
2026-01-05 08:49:39	ListBuckets
2026-01-05 08:49:58	ListAttachedUserPolicies
2026-01-05 08:50:15	CreateAccessKey
2026-01-05 08:50:38	ListAccessKeys

Which other IAM user did the adversary discover and backdoor?

lars.andersen

```
index=task5
| table _time eventName requestParameters.userName responseElements.accessKey.userName
```

it is who check ListAttachedUserPolicies

Task 6 - Denial of Wallet Attacks

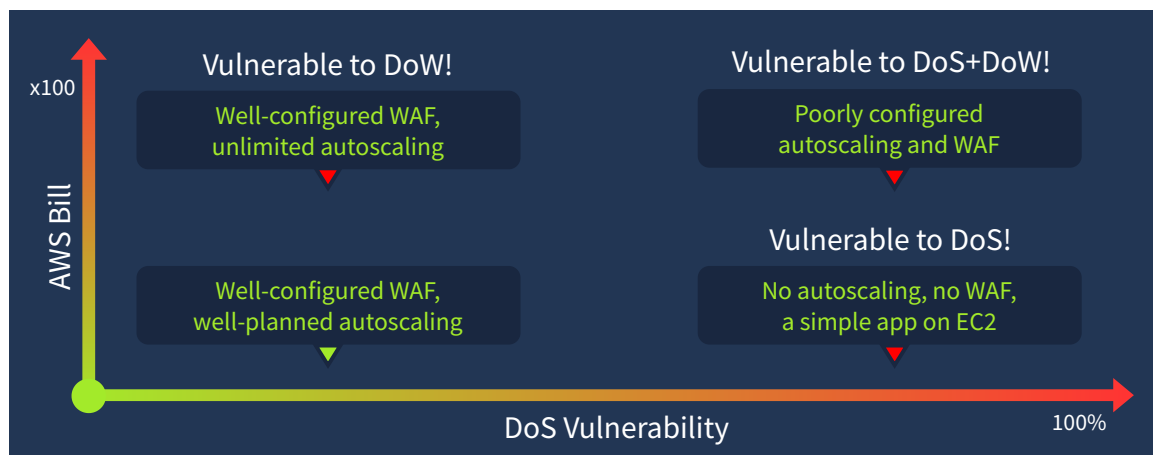
Denial of Wallet Attacks: Unlike traditional Denial of Service (DoS) attacks that aim to take services offline, Denial of Wallet (DoW) attacks exploit pay-as-you-go cloud pricing models (e.g., serverless execution, autoscaling, WAF requests). Adversaries continuously trigger automated usage to artificially inflate compute,

bandwidth, or inspection costs over time, attempting to drain an organization's financial resources rather than disrupting uptime.

DoW Attack Examples

Attack on S3 Buckets: Amazon S3 pricing is heavily driven by object request volume (`GetObject` , `PutObject`) and outbound data transfer. Attackers execute Denial of Wallet (DoW) attacks on S3 by repeatedly flooding a bucket with high-volume requests or continuously downloading large files (such as PDFs or videos) to rapidly inflate request charges and egress bandwidth fees.

Attack on Compute Workloads: Denial of Wallet (DoW) attacks on compute workloads often present as unintended side effects of Denial of Service (DoS) mitigations. When attackers flood an application to disrupt service, AWS Auto Scaling automatically provisions additional EC2 instances to absorb the traffic load. While this mechanism preserves application availability, the unexpected resource expansion can result in massive AWS bills—effectively shifting a DoS attempt into a costly DoW scenario.



SOC Team and Mitigation: Same as with DoS attacks, the SOC team can help with detection and triage, but not full remediation. All you can do is detect a spike of requests, block offensive IPs (which is a short-term response), and report the risks to your IT team. However, what you should know is what to say when IT team has no idea how to address the risks. Even while exact steps require strong AWS expertise, the general approach for any cloud is:

1. **Validate if the resource under attack is expected to be public:** It is possible that it shouldn't have been public in the first place.

2. **Block exploits, known-bad IPs, and user-agents with WAF:** In most clouds, blocked requests are much cheaper than allowed.
3. **Cache server responses and rate limit user requests with CDN:** For example, you can cache images in S3 with CloudFront CDN.
4. **Finally, optimize the compute work (CPU, RAM, network usage):** The faster your app is, the lower the amount you pay for compute services.

Answer the questions below

| What does the acronym DoW stand for?

Denial of Wallet

| Should you monitor DoW with the same effort as DoS? (Yea/Nay)

Yea

Task 7 - Conclusion

Unnecessary internet exposure combined with weak or missing authentication is the primary root cause driving cloud security incidents—including Denial of Wallet (DoW) attacks, data exfiltration, and initial intrusions. Detecting cloud resource exposure and leaked credentials should serve as top priorities for SOC teams across all major providers (AWS, Azure, and Google Cloud), as services like Azure Blob Storage face identical exposure risks.

By applying the Pareto principle (focusing SOC monitoring efforts on the ~20% of high-impact vectors, such as S3/Blob storage, EC2/VM security groups, and IAM permissions), security teams can effectively detect up to 80% of common cloud threats using CloudTrail and telemetry logs.